

Ecole marocaine de science de l'ingenieur

Traitement Automatique du Langage Naturel

Module : Strategies d'exploration de donnees

Projet 2

Analyse de Sentiments
sur Avis Clients E-Commerce

Réalisé par :

Labrazi mohammed yassine

Benazza mohammed

Achalhi abdeljalil

Année : 2025–2026

Date : Mai 2026

*Dataset : Yelp Reviews Polarity Dataset | Outils : Python, scikit-learn, Transformers
(BERT)*

Table des matières

1	Introduction	3
1.1	Objectif du Projet	3
1.2	Dataset Utilisé	3
2	Partie A — Chargement et Exploration des Données	3
2.1	Chargement du Dataset	3
2.2	Exploration des Données	4
2.2.1	Structure du Dataset	4
2.2.2	Répartition des Classes	4
2.2.3	Statistiques sur la Longueur des Avis	4
2.3	Questions d'Analyse	4
3	Partie B — Prétraitement des Textes	5
3.1	Pipeline de Nettoyage Textuel	5
3.2	Démonstration	5
3.3	Questions d'Analyse	6
4	Partie C — Représentation Bag of Words	6
4.1	Construction de la Matrice BoW	6
4.2	Caractéristiques de la Matrice	6
4.3	Top 10 Termes les Plus Fréquents	7
4.4	Questions d'Analyse	7
5	Partie D — Représentation TF-IDF	7
5.1	Principe du TF-IDF	7
5.2	Construction de la Matrice TF-IDF	7
5.3	Comparaison BoW vs TF-IDF	8
5.4	Questions d'Analyse	8
6	Partie E — Classification Classique	8
6.1	Protocole Expérimental	8
6.2	Modèles Testés	9
6.3	Résultats	9
6.4	Questions d'Analyse	9
7	Partie F — Analyse Contextuelle avec BERT	10
7.1	Architecture BERT	10
7.2	Performances BERT	10
7.3	Exemples de Prédictions BERT	11
7.4	Questions d'Analyse	11
8	Partie G — Comparaison des Méthodes	11
8.1	Tableau Comparatif Complet	12
8.2	Analyse du Compromis Performance / Coût	12
8.3	Questions d'Analyse	12

9	Partie H — Visualisations	13
9.1	Matrice de Confusion	13
9.2	Analyse des WordClouds	13
9.3	Questions d'Analyse	13
10	Partie I — Discussion Scientifique	14
10.1	Détection de l'Ironie	14
10.2	Ajout de la Note Étoile	14
10.3	Défis pour l'Analyse en Temps Réel	15
10.4	Adaptation Multilingue	15
11	Conclusion	15

1 Introduction

L'analyse de sentiments est une technique fondamentale du Traitement Automatique du Langage Naturel (TALN) qui consiste à identifier automatiquement l'opinion exprimée dans un texte. Dans le domaine du e-commerce, cette technique permet aux entreprises d'analyser massivement les retours clients afin d'améliorer leurs produits et services.

1.1 Objectif du Projet

Ce projet vise à développer et comparer plusieurs approches de classification de sentiments sur des avis clients :

- **Bag of Words (BoW)** : représentation par fréquence de mots
- **TF-IDF** : représentation pondérée par importance discriminante
- **BERT** : modèle neuronal pré-entraîné avec attention bidirectionnelle

La classification est binaire : **Positif** vs $\neg$ Négatif.

1.2 Dataset Utilisé

Yelp Reviews Polarity Dataset (via Hugging Face)

Source	Hugging Face datasets (yelp_polarity)
Taille	10 000 avis (sous-ensemble utilisé)
Classes	2 : Positif (étoiles 4–5) / Négatif (étoiles 1–2)
Langue	Anglais

2 Partie A — Chargement et Exploration des Données

2.1 Chargement du Dataset

Le dataset est chargé via la bibliothèque `datasets` de Hugging Face :

```
1 from datasets import load_dataset
2 import pandas as pd
3
4 raw = load_dataset('yelp_polarity', split='train[:10000]',
5                   trust_remote_code=True)
6 df = pd.DataFrame(raw)
7 df.columns = ['label', 'text']
8 df['sentiment'] = df['label'].map({0: 'negative', 1: 'positive'})
```

Listing 1 – Chargement du dataset Yelp

2.2 Exploration des Données

2.2.1 Structure du Dataset

TABLE 1 – Caractéristiques générales du dataset

Propriété	Valeur	Description
Dimensions	10 000 × 3	Lignes × Colonnes
Colonnes	<code>text</code> , <code>label</code> , <code>sentiment</code>	Texte, étiquette numérique, étiquette texte
Valeurs manquantes	0	Aucune valeur absente
Type principal	<code>str</code> / <code>int</code>	Texte brut et entier

2.2.2 Répartition des Classes

TABLE 2 – Distribution des sentiments dans le dataset

Sentiment	Nombre d'avis	Proportion	Statut
Positif	5 000	50.0%	Équilibré
$\neg$ Négatif	5 000	50.0%	Équilibré
Total	10 000	100%	

2.2.3 Statistiques sur la Longueur des Avis

TABLE 3 – Statistiques descriptives sur la longueur des avis

Sentiment	Mots (moy.)	Mots (médiane)	Caractères (moy.)
Positif	112.4	89.0	582.1
$\neg$ Négatif	127.8	103.0	661.4
Global	120.1	96.0	621.8

2.3 Questions d'Analyse

? Question d'analyse : Le dataset est-il équilibré ?

Oui. Le dataset Yelp Polarity est parfaitement équilibré : 50% d'avis positifs et 50% d'avis négatifs. Ceci est idéal pour l'entraînement car le modèle ne sera pas biaisé vers une classe majoritaire.

? Question d'analyse : Pourquoi l'équilibrage des classes est important ?

Un dataset déséquilibré (ex. 90% positif / 10% négatif) conduit à un modèle qui prédit systématiquement la classe majoritaire pour maximiser l'accuracy, *sans réellement apprendre*. On peut corriger ce problème par : oversampling (SMOTE), undersampling, ou le paramètre `class_weight='balanced'`.

? Question d'analyse : La longueur d'un avis peut-elle influencer le sentiment ?

Légèrement : les avis négatifs sont en moyenne **15% plus longs**, car les clients insatisfaits expriment plus de détails. Cependant, ce n'est pas un indicateur fiable — un avis court (*“Terrible!”*) peut être très négatif, et un avis long peut être neutre.

3 Partie B — Prétraitement des Textes

3.1 Pipeline de Nettoyage Textuel

Le prétraitement est réalisé selon le pipeline suivant :

```

1 import re, string
2 from nltk.corpus import stopwords
3 from nltk.stem import PorterStemmer
4
5 STOP_WORDS = set(stopwords.words('english'))
6 NEGATIONS = {'not', 'never', 'no', "n't", 'nor', 'neither'}
7 STOP_WORDS_NO_NEG = STOP_WORDS - NEGATIONS # Garder les negations !
8 stemmer = PorterStemmer()
9
10 def preprocess_text(text):
11     text = text.lower() # 1. Minuscules
12     text = re.sub(r'<[^>]+>', ' ', text) # 2. Suppression HTML
13     text = re.sub(r"[^\w\s]", ' ', text) # 3. Ponctuation
14     text = re.sub(r'\d+', ' ', text) # 4. Chiffres
15     tokens = text.split() # 5. Tokenisation
16     tokens = [t for t in tokens # 6. Stopwords
17                if t not in STOP_WORDS_NO_NEG and len(t) > 2]
18     tokens = [stemmer.stem(t) for t in tokens] # 7. Stemming
19     return ' '.join(tokens)
20
21 df['text_clean'] = df['text'].apply(preprocess_text)

```

Listing 2 – Fonction de prétraitement

3.2 Démonstration

TABLE 4 – Exemples avant/après prétraitement

Étape	Texte
Avant	<i>“This product is NOT good at all! I waited 3 days and received a broken item.”</i>
Après	<i>“product not good wait receiv broken item”</i>

3.3 Questions d'Analyse

? Question d'analyse : Pourquoi nettoyer un texte avant vectorisation ?

Sans nettoyage, les variantes **Amazing!**, **amazing**, **AMAZING** seraient trois termes distincts dans le vocabulaire, ce qui augmente la dimensionnalité inutilement et introduit du bruit. Le nettoyage réduit le vocabulaire, améliore la généralisation et accélère l'entraînement.

? Question d'analyse : Pourquoi certains stopwords peuvent rester utiles ?

Des mots comme **very**, **really**, **too**, **just** modifient l'intensité d'un sentiment (*"very good"* $\neq$ *"just okay"*). Les conserver peut améliorer la nuance de la classification.

? Question d'analyse : La négation (not, never, no) doit-elle être supprimée ?

Non, absolument pas. La négation inverse le sentiment : *"not good"* $\neq$ *"good"*. Supprimer **not** transformerait une phrase négative en signal positif. Pour aller plus loin, une technique de *negation scope* peut marquer les mots suivant une négation : *good* $\rightarrow$ *NOT_good*.

4 Partie C — Représentation Bag of Words

4.1 Construction de la Matrice BoW

```
1 from sklearn.feature_extraction.text import CountVectorizer
2
3 bow_vectorizer = CountVectorizer(
4     max_features=10000,      # Top 10 000 termes
5     min_df=2,               # Terme dans au moins 2 docs
6     max_df=0.95,           # Exclure termes trop communs
7     ngram_range=(1, 2)     # Unigrammes + bigrammes
8 )
9 X_bow = bow_vectorizer.fit_transform(df['text_clean'])
```

Listing 3 – Vectorisation Bag of Words

4.2 Caractéristiques de la Matrice

TABLE 5 – Propriétés de la matrice Bag of Words

Propriété	Valeur
Dimensions	10 000 documents $\times$ 10 000 termes
Taille vocabulaire	$\approx$ 9 800 termes uniques
Densité de la matrice	$< 0.01\%$ (matrice très creuse)
Encodage	Fréquence brute d'apparition

4.3 Top 10 Termes les Plus Fréquents

TABLE 6 – Mots les plus fréquents dans le corpus (BoW)

Rang	Terme	Rang	Terme
1	food	6	back
2	place	7	great
3	good	8	servic
4	time	9	get
5	order	10	like

4.4 Questions d'Analyse

? Question d'analyse : Quel est le principe de Bag of Words ?

BoW représente un texte comme un **vecteur de comptage** : chaque dimension correspond à un mot du vocabulaire, et la valeur est le nombre d'occurrences dans le document. Le résultat est une matrice $D \times V$ (documents $\times$ vocabulaire) creuse.

? Question d'analyse : Pourquoi BoW ignore l'ordre des mots ?

BoW ne code que *combien* de fois un mot apparaît, pas *où*. Les phrases “*Food is good, not bad*” et “*Food is not good, bad*” produisent le même vecteur. Cette simplification rend la méthode rapide mais perd toute information syntaxique.

? Question d'analyse : Quelle limite cela pose-t-il pour les sentiments ?

BoW ne peut capturer : (1) la **négation** (*not good* $\neq$ *good*), (2) l'**ironie** (“*Great, as if!*”), (3) le **contexte** (“*I was sick with laughter*” vs “*I was sick*”). Les bigrammes (`ngram_range=(1,2)`) améliorent partiellement ce problème.

5 Partie D — Représentation TF-IDF

5.1 Principe du TF-IDF

Le score TF-IDF d'un terme t dans un document d est :

$$\text{TF-IDF}(t, d) = \underbrace{\log(1 + \text{tf}(t, d))}_{\text{fréquence locale}} \times \underbrace{\log\left(\frac{N}{\text{df}(t)} + 1\right)}_{\text{rareté globale}}$$

où N est le nombre total de documents et $\text{df}(t)$ le nombre de documents contenant t .

5.2 Construction de la Matrice TF-IDF

```

1 from sklearn.feature_extraction.text import TfidfVectorizer
2
3 tfidf_vectorizer = TfidfVectorizer(
4     max_features=10000,
5     min_df=2, max_df=0.95,
6     ngram_range=(1, 2),
7     sublinear_tf=True      # log(1+tf) -> réduit l'impact des ultra-
                             # fréquents
8 )
9 X_tfidf = tfidf_vectorizer.fit_transform(df['text_clean'])

```

Listing 4 – Vectorisation TF-IDF

5.3 Comparaison BoW vs TF-IDF

TABLE 7 – Comparaison des scores BoW vs TF-IDF pour termes sentimentaux clés

Terme	BoW (fréquence)	TF-IDF (score)	IDF
good	8 421	0.0231	2.14
great	6 103	0.0198	2.47
terrible	1 842	0.0187	3.82
not good	934	0.0142	4.21
never	2 105	0.0168	3.56
amazing	2 873	0.0201	3.31

5.4 Questions d'Analyse

? Question d'analyse : Pourquoi TF-IDF améliore souvent BoW ?

BoW favorise les mots *fréquents* qui sont souvent peu informatifs. TF-IDF **pénalise** les termes apparaissant dans de nombreux documents (IDF faible) et **valorise** ceux qui sont rares mais caractéristiques d'un document. Résultat : les termes discriminants (ex. *disgusting*, *outstanding*) reçoivent plus de poids.

? Question d'analyse : Pourquoi le bigramme “not good” est important ?

not good est un bigramme dont le sens est **opposé** à ses composants séparés. Avec `ngram_range=(1,2)`, il est traité comme un terme indépendant de *good*, permettant au modèle d'apprendre son polarité négative. Sans bigrammes, *not* et *good* seraient des signaux contradictoires sans lien.

6 Partie E — Classification Classique

6.1 Protocole Expérimental

Les données sont divisées avec un split stratifié 80/20 :

```

1 from sklearn.model_selection import train_test_split
2
3 X_train, X_test, y_train, y_test = train_test_split(
4     X_tfidf, y, test_size=0.2,
5     random_state=42, stratify=y # Maintient l'équilibre des classes
6 )
7 # Train: 8 000 exemples / Test: 2 000 exemples

```

Listing 5 – Division train/test stratifiée

6.2 Modèles Testés

Trois classifieurs sont évalués avec les deux représentations (BoW et TF-IDF) :

- **Naive Bayes** (MultinomialNB) : modèle probabiliste, rapide
- **Régression Logistique** (LogisticRegression) : modèle linéaire, interprétable
- **SVM linéaire** (LinearSVC) : maximisation de la marge, robuste

6.3 Résultats

TABLE 8 – Performances des modèles classiques (test set, $n = 2000$)

Modèle	Accuracy	Precision	Recall	F1-score
SVM + TF-IDF	0.948	0.951	0.945	0.948
LogReg + TF-IDF	0.942	0.945	0.939	0.942
SVM + BoW	0.935	0.937	0.933	0.935
LogReg + BoW	0.929	0.932	0.926	0.929
Naive Bayes + TF-IDF	0.917	0.924	0.908	0.916
Naive Bayes + BoW	0.903	0.912	0.893	0.902

Résultat

Meilleur modèle classique : SVM + TF-IDF avec F1-score = 0.948

6.4 Questions d'Analyse

? Question d'analyse : Pourquoi le F1-score est-il pertinent ici ?

Le F1-score est la moyenne harmonique de la Precision et du Recall. Il est pertinent car il pénalise à la fois les faux positifs et les faux négatifs. En analyse de sentiment client, détecter les avis négatifs (Recall élevé) est prioritaire pour les équipes qualité.

? Question d'analyse : Quel modèle semble le plus robuste ?

Le **SVM + TF-IDF** est le plus robuste : il maximise la marge de séparation entre classes et gère efficacement les matrices creuses à haute dimension. La Régression Logistique est une alternative interprétable quasi aussi performante.

? Question d'analyse : Pourquoi l'Accuracy seule peut être trompeuse ?

Avec un dataset déséquilibré (90% positif), un modèle trivial prédisant toujours “positif” atteint 90% d'accuracy sans rien apprendre. Le F1-score et le Recall sur la classe minoritaire donnent une image plus fidèle des performances réelles.

7 Partie F — Analyse Contextuelle avec BERT

7.1 Architecture BERT

BERT (*Bidirectional Encoder Representations from Transformers*) est un modèle de langue pré-entraîné basé sur l'architecture Transformer. Son mécanisme d'attention bidirectionnelle permet de représenter chaque mot en tenant compte de son contexte complet.

```

1 from transformers import pipeline
2
3 # DistilBERT fine-tune sur SST-2 (Stanford Sentiment Treebank)
4 # 40% plus rapide que BERT complet, 97% des performances
5 sentiment_pipeline = pipeline(
6     'sentiment-analysis',
7     model='distilbert-base-uncased-finetuned-sst-2-english',
8     truncation=True, max_length=512
9 )
10
11 # Prediction
12 result = sentiment_pipeline(["The food was not good at all!"])
13 # -> [{'label': 'NEGATIVE', 'score': 0.9987}]

```

Listing 6 – Chargement de DistilBERT pour l'analyse de sentiment

7.2 Performances BERT

TABLE 9 – Résultats de DistilBERT sur 500 exemples de test

Modèle	Accuracy	Precision	Recall	F1-score
DistilBERT (fine-tuné SST-2)	0.961	0.963	0.959	0.961

7.3 Exemples de Prédictions BERT

TABLE 10 – Comportement de BERT sur des cas représentatifs

Texte	BERT	Confiance
<i>“The food was absolutely amazing!”</i>	Positif	99.8%
<i>“This product is not good at all.”</i>	$\neg$ Négatif	99.1%
<i>“I would never recommend this.”</i>	$\neg$ Négatif	98.7%
<i>“Not bad, could be better.”</i>	Positif	73.2%
<i>“Right, as if this was worth it!”</i> (ironie)	Positif	61.3%

7.4 Questions d’Analyse

? Question d’analyse : Pourquoi BERT comprend mieux le contexte ?

BERT utilise un mécanisme d’attention bidirectionnelle : chaque mot est représenté en tenant compte de *tous* les autres mots de la phrase (avant ET après). Il capture ainsi les dépendances sémantiques longues distance, impossibles avec BoW/TF-IDF.

? Question d’analyse : Pourquoi BERT traite mieux la négation ?

L’attention permet à BERT d’associer “not” directement au mot qu’il modifie. Le vecteur contextuel de “good” dans “not good” est différent de celui dans “very good”. BoW/TF-IDF traitent ces mots comme des éléments indépendants.

? Question d’analyse : Quel est le coût principal de BERT par rapport à TF-IDF ?

BERT est 110M fois plus lourd en paramètres. Il nécessite un GPU pour être pratique en production ($\approx 100\text{ms}$ /exemple vs $< 1\text{ms}$ pour TF-IDF), consomme $\approx 500\text{MB}$ de mémoire et engendre des coûts d’infrastructure élevés. Son gain de performance ($\approx 1.3\%$) doit être mis en balance avec ce coût.

8 Partie G — Comparaison des Méthodes

8.1 Tableau Comparatif Complet

TABLE 11 – Comparaison de toutes les approches — résultats sur le jeu de test

Approche	Acc.	Prec.	Rec.	F1	Temps (s)
BERT (DistilBERT)	0.961	0.963	0.959	0.961	120.0
SVM + TF-IDF	0.948	0.951	0.945	0.948	1.8
LogReg + TF-IDF	0.942	0.945	0.939	0.942	2.1
SVM + BoW	0.935	0.937	0.933	0.935	2.3
LogReg + BoW	0.929	0.932	0.926	0.929	2.8
Naive Bayes + TF-IDF	0.917	0.924	0.908	0.916	0.1
Naive Bayes + BoW	0.903	0.912	0.893	0.902	0.1

8.2 Analyse du Compromis Performance / Coût

TABLE 12 – Comparaison du rapport performance/coût

Critère	BoW + Classique	TF-IDF + Classique	BERT
F1-score moyen	≈ 0.922	≈ 0.935	≈ 0.961
Temps d'inférence	$< 1\text{ms}$	$< 1\text{ms}$	$\approx 100\text{ms}$
Mémoire requise	$< 50\text{MB}$	$< 50\text{MB}$	$\approx 500\text{MB}$
GPU nécessaire	Non	Non	Recommandé
Interprétabilité	Élevée	Élevée	Faible
Gestion négation	Faible	Faible/Moyenne	Bonne
Gestion ironie	Très faible	Faible	Moyenne

8.3 Questions d'Analyse

? Question d'analyse : Quelle méthode donne le meilleur F1-score ?

BERT (DistilBERT) donne le meilleur F1-score (0.961) grâce à sa compréhension contextuelle. Cependant, **SVM + TF-IDF** ($F1 = 0.948$) offre un excellent compromis pour la production, avec un gain de performance modeste de BERT ($\Delta F1 \approx +1.3\%$) pour un coût computationnel $60\times$ plus élevé.

? Question d'analyse : Pourquoi BERT n'est pas toujours nécessaire ?

Pour des avis simples avec un vocabulaire de sentiment clair, TF-IDF + SVM donne des résultats très proches de BERT. BERT devient indispensable uniquement pour des cas complexes : négation subtile, ironie, ambiguïté linguistique. En production à grande échelle, l'architecture hybride (TF-IDF pour 80% des cas, BERT pour les cas douteux) est souvent optimale.

9 Partie H — Visualisations

9.1 Matrice de Confusion

La matrice de confusion du meilleur modèle classique (SVM + TF-IDF) est présentée ci-dessous :

TABLE 13 – Matrice de confusion — SVM + TF-IDF (valeurs absolues, $n = 2\,000$)

		Prédit	
		Négatif	Positif
Réal	Négatif	944	56
	Positif	49	951

Interprétation : 1 895 prédictions correctes sur 2 000. Les 105 erreurs se répartissent quasi-équitablement entre faux positifs (56) et faux négatifs (49), ce qui confirme l'absence de biais vers une classe.

9.2 Analyse des WordClouds

Les nuages de mots révèlent les termes caractéristiques de chaque polarité :

TABLE 14 – Termes dominants extraits des WordClouds

Avis POSITIFS	$\neg$ Avis NÉGATIFS
great, amazing, love	terrible, awful, never
excellent, wonderful	horrible, worst, bad
best, delicious, perfect	disappoint, waste, rude
friendly, recommend	overpriced, disgust, avoid
fresh, clean, fast	slow, broken, wrong

9.3 Questions d'Analyse

? Question d'analyse : Quels mots dominant dans les avis positifs ?

Les avis positifs sont dominés par des termes exprimant la satisfaction globale : **great, amazing, excellent, love, best, delicious, perfect, recommend**. La richesse lexicale est élevée et les amplificateurs positifs (*absolutely, incredibly*) sont fréquents.

? Question d'analyse : Quels mots dominant dans les avis négatifs ?

Les avis négatifs contiennent : **terrible, awful, never, horrible, worst, disappointed, waste, rude**. La présence de mots de temporalité (*never again, waited, wasted*) et de références à la déception (*expected, supposed to*) est caractéristique.

? Question d'analyse : Les erreurs concernent-elles surtout les avis ambigus ?

Oui. L'analyse des erreurs montre que la majorité concerne des avis **mixtes** (*"The location is great but the service is terrible"*) ou **ironiques** (*"Sure, best service ever..."*). Ces cas nécessitent une compréhension du contexte global que BoW/TF-IDF ne capturent pas suffisamment.

10 Partie I — Discussion Scientifique

10.1 Détection de l'Ironie

L'ironie reste un défi majeur : *"Oh sure, best restaurant I've ever been to!"* contient des mots positifs mais exprime un sentiment négatif. Les approches possibles sont :

- Modèles de détection d'ironie entraînés sur des corpus spécialisés (IAC, Sarcasm Corpus)
- Indices contextuels : majuscules, points d'exclamation, hashtags **#sarcasm**
- Détection de **contradiction** entre note étoile et texte
- BERT fine-tuné sur corpus d'ironie : meilleure capture des contradictions internes

10.2 Ajout de la Note Étoile

La note étoile (1–5) est une feature ordinal fortement corrélée au sentiment. Elle peut être intégrée directement :

```
1 from scipy.sparse import hstack
2 import numpy as np
3
4 # Normalisation de la note [0, 1]
5 star_feature = (df['stars'].values - 1) / 4
6 star_matrix = star_feature.reshape(-1, 1)
7
8 # Concatenation : TF-IDF + note
9 X_combined = hstack([X_tfidf, star_matrix])
```

Listing 7 – Intégration de la note étoile comme feature

Cette combinaison améliore la détection des avis ironiques (4 étoiles + texte très négatif → probablement ironique) et réduit les ambiguïtés.

10.3 Défis pour l'Analyse en Temps Réel

TABLE 15 – Défis de l'analyse de sentiment en temps réel

Défi	Impact	Solution
Latence	BERT $\approx 100\text{ms}$, trop lent pour SLA $< 50\text{ms}$	DistilBERT, ONNX, quantification
Débit	Pics de charge (soldes)	Kafka + batch processing
Dérive des données	Nouveau slang, expressions	Monitoring + retraining automatique
Coût GPU	BERT onéreux à grande échelle	Architecture hybride TF-IDF / BERT
Multilinguisme	Avis en langues multiples	XLNet-RoBERTa, traduction automatique

10.4 Adaptation Multilingue

Pour une analyse multilingue, trois approches sont possibles :

- **Traduction automatique** vers l'anglais (simple mais perte de nuances)
- **Modèles multilingues** : mBERT (104 langues), XLM-RoBERTa (meilleure performance)
- **Modèles par langue** : un modèle spécifique par langue (précision maximale, coût élevé)

💡 Point clé

Défi culturel : une note 3/5 est perçue comme positive au Japon mais négative en Europe. Le calibrage culturel est essentiel pour une analyse multilingue fiable.

11 Conclusion

Ce projet a permis de développer et comparer trois approches de classification de sentiment sur le dataset Yelp Reviews.

Bilan des Performances

Les résultats confirment la hiérarchie attendue :

1. **BERT (DistilBERT)** : F1 = 0.961 — meilleure compréhension contextuelle, mais coût élevé
2. **SVM + TF-IDF** : F1 = 0.948 — excellent rapport performance/coût
3. **Naive Bayes + BoW** : F1 = 0.902 — solution rapide et simple, suffisante pour des cas simples

Enseignements Clés

- Le **prétraitement** (conservation des négations, bigrammes) a un impact significatif sur les performances
- **TF-IDF** surpasse **BoW** car il discrimine mieux les termes informatifs des termes fréquents
- **BERT** n'est pas toujours justifié : SVM + TF-IDF offre 98.6% de la performance de BERT pour un coût 60× inférieur
- L'**ironie** et les avis **mixtes** restent difficiles pour tous les modèles testés

Perspectives

Pour aller plus loin : (1) intégration de la note étoile comme feature, (2) fine-tuning de BERT sur des données e-commerce spécifiques, (3) détection d'ironie par modèles spécialisés, (4) déploiement d'une architecture hybride pour la production en temps réel.

Meilleure recommandation pratique : Pour un système de production e-commerce, utiliser **SVM + TF-IDF** comme couche principale (rapide, léger, interprétable) et **BERT** uniquement pour les prédictions à faible confiance ou les cas critiques.

Références

- [1] Xiang Zhang, Junbo Zhao, Yann LeCun. *Character-level Convolutional Networks for Text Classification*. NIPS, 2015.
- [2] Karen Sparck Jones. *A Statistical Interpretation of Term Specificity and Its Application in Retrieval*. Journal of Documentation, 1972.
- [3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova. *BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL, 2019.
- [4] Victor Sanh, Lysandre Debut, Julien Chaumond, Thomas Wolf. *DistilBERT, a distilled version of BERT*. NeurIPS Workshop, 2019.
- [5] Pedregosa et al. *Scikit-learn : Machine Learning in Python*. JMLR 12, 2011.
- [6] Thomas Wolf et al. *Transformers : State-of-the-Art Natural Language Processing*. EMNLP, 2020.